



Listas, Tuplas, Conjuntos y Algoritmos de Ordenamiento

Organiza, almacena y ordena tus datos

Skill: Introducción a la Programación

Docente: Jhonathan Arley Peñaloza Sierra

Campuslands · 2026

Objetivos de Aprendizaje

- 1 Comprender qué son las estructuras de datos y por qué son importantes
- 2 Crear y manipular listas en Python usando sus métodos principales
- 3 Entender las tuplas: cuándo usarlas y sus diferencias con las listas
- 4 Conocer los conjuntos (sets) y sus operaciones
- 5 Conocer los algoritmos de ordenamiento más comunes

¿Qué es una Estructura de Datos?



Piensa en un EDIFICIO...

Un edificio organiza espacios:

- Apartamentos para vivir
- Oficinas para trabajar
- Locales para vender

Cada espacio tiene un propósito específico.



Organiza datos en el programa:

- Listas para secuencias ordenadas
- Tuplas para datos que no cambian
- Conjuntos para valores únicos

Cada estructura tiene un uso ideal.



Definición:

Una estructura de datos es una forma organizada de almacenar y manipular datos para que puedan ser utilizados eficientemente por el programa.

¿Qué es una Lista?



Piensa en una lista de compras...

En programación, una lista es un contenedor que almacena múltiples elementos en un orden específico. Cada elemento tiene una posición (índice) que empieza desde 0.

```
cajon = [ 7, "Hola", True, 3.5 ]
```

Índice 0
7

Índice 1
"Hola"

Índice 2
True

Índice 3
3.5

Características principales:

- ✓ Ordenadas
- ✓ Indexadas (desde 0)
- ✓ Mutables (se pueden cambiar)
- ✓ Pueden mezclar tipos de datos

Crear Listas en Python

```
# Lista vacía
mi_lista = []

# Lista con elementos de diferentes tipos
elementos = [1, "Python", 3.14, True]

# Lista anidada (listas dentro de listas)
lista_anidada = [[1,2,3], ["a","b","c"]]
```

Recuerda:

- Se crean con corchetes []
- Elementos separados por comas
- Pueden contener cualquier tipo
- Pueden contener otras listas



¿Para qué sirven?

Organización, acceso rápido por índice, almacenar colecciones de datos, y facilitar tareas repetitivas con bucles.

Métodos de Listas: Agregar Elementos

`append(item)`

Agrega UN elemento al final.

```
frutas = ["manzana", "pera"]
frutas.append("naranja")
# ['manzana', 'pera', 'naranja']
```

`extend(iterable)`

Agrega VARIOS elementos de otra lista.

```
nums = [1, 2, 3]
nums.extend([4, 5, 6])
# [1, 2, 3, 4, 5, 6]
```

`insert(pos, item)`

Inserta un elemento en una posición específica.

```
colores = ["rojo", "azul"]
colores.insert(1, "verde")
# ['rojo', 'verde', 'azul']
```

Métodos de Listas: Eliminar Elementos

`remove(item)`

Elimina la primera aparición del elemento.

```
nombres = ["Ana","Luis","Ana"]
nombres.remove("Ana")
# ['Luis', 'Ana']
```

`pop([index])`

Elimina y devuelve el elemento en el índice. Sin índice, elimina el último.

```
letras = ["a","b","c"]
letras.pop() # devuelve "c"
# ['a', 'b']
```

`clear()`

Elimina TODOS los elementos de la lista.

```
nums = [1, 2, 3]
nums.clear()
# []
```

Métodos de Listas: Buscar y Contar

`index(item)`

Devuelve el índice de la primera aparición.

```
animales = ["gato", "perro", "pez"]  
animales.index("perro") # → 1
```

`count(item)`

Cuenta cuántas veces aparece un elemento.

```
vocales = ["a", "e", "i", "o", "u", "a"]  
vocales.count("a") # → 2
```

`in (operador)`

Verifica si un elemento está en la lista.

```
colores = ["rojo", "azul", "verde"]  
"azul" in colores # → True
```

`any(iterable)`

True si al menos un elemento es verdadero.

```
nums = [0, 0, 1]  
any(nums) # → True
```

Métodos de Listas: Ordenar e Invertir

sort()

Ordena los elementos de la lista.

```
# Ascendente (por defecto)
nums = [5, 3, 8, 1]
nums.sort()
# [1, 3, 5, 8]

# Descendente
nums.sort(reverse=True)
# [8, 5, 3, 1]

# Por longitud de palabra
palabras = ["manzana", "kiwi", "pera"]
palabras.sort(key=len)
# ['kiwi', 'pera', 'manzana']
```

reverse()

Invierte el orden de los elementos.

```
palabras = ["uno", "dos", "tres"]
palabras.reverse()
# ['tres', 'dos', 'uno']
```

copy()

Crea una copia independiente de la lista.

```
nueva = lista.copy()
# nueva es independiente de lista
```

Listas Anidadas (Listas dentro de Listas)

Las listas pueden contener otras listas. Son ideales para representar tablas o matrices.

```
matriz = [  
    [1, 2, 3],    # fila 0  
    [4, 5, 6],    # fila 1  
    [7, 8, 9]     # fila 2  
]  
  
# Acceder al 5 (fila 1, columna 1)  
print(matriz[1][1])  # → 5
```

 ¿Cómo acceder?

`matriz[fila][columna]`

`matriz[0][0] → 1`

`matriz[1][1] → 5`

`matriz[2][2] → 9`

`matriz[0][2] → 3`



Recorrer con for anidado:

for fila in matriz:

for elemento in fila:

print(elemento)



Actividad Práctica: Listas

1. Crea una lista de 5 frutas e imprime la que está en el índice 2
2. Agrega "uva" al final con `append()` e inserta "fresa" en la posición 1
3. Elimina la primera fruta con `pop(0)` e imprime la lista resultante
4. Ordena la lista alfabéticamente con `sort()` e imprímela
5. Crea una lista anidada 3×3 y accede al elemento central



Tiempo: 10 minutos

Tuplas: Datos que NO Cambian

Una tupla es como una lista, pero INMUTABLE: una vez creada, no se puede modificar.

Lista [] → Mutable ✓

```
lista = [1, 2, 3]
lista[0] = 99 # ✓ Funciona
# [99, 2, 3]
```

Tupla () → Inmutable ✗

```
tupla = (1, 2, 3)
tupla[0] = 99 # ✗ TypeError!
# No se puede cambiar
```

Crear tuplas:

```
tupla = (1, 2, 3)           # Con paréntesis
tupla = 1, 2, 3             # Sin paréntesis (también válido)
tupla_un_elem = (2,)        # Un solo elemento: necesita la coma
lista_a_tupla = tuple([1,2,3]) # Convertir lista a tupla
```

Métodos y Operaciones de Tuplas

Las tuplas tienen solo 2 métodos (porque son inmutables):

count(obj)

Cuenta cuántas veces aparece un elemento.

```
t = (1, 1, 1, 3, 5)
t.count(1) # → 3
```

index(obj)

Devuelve el índice donde se encuentra.

```
t = (7, 7, 7, 3, 5)
t.index(5) # → 4
```

Operaciones útiles:

```
# Iterar tupla
for t in (1, 2, 3): print(t)

# Desempaquetar valores
x, y, z = (10, 20, 30) # x=10, y=20, z=30

# Convertir tupla ↔ lista
mi_lista = list(mi_tupla) # Tupla a lista (para modificar)
mi_tupla = tuple(mi_lista) # Lista a tupla (para proteger)
```

Ejemplo: Librería con Lista de Tuplas

 Cada libro es una tupla (título, autor, género) dentro de una lista:

```
libros = [("El Perfume","Suskind","Misterio"),
          ("La Metamorfosis","Kafka","Novela"),
          ("Misery","Stephen King","Misterio"),
          ("Harry Potter","J.K. Rowling","Novela")]

# Buscar libros por género
genero = "Misterio"
resultado = []
for libro in libros:
    if libro[2] == genero:
        resultado.append(libro[0])
print(resultado)  # ['El Perfume','Misery']
```

Conjuntos (Sets): Valores Únicos

Un conjunto (set) es una colección de elementos ÚNICOS y SIN ORDEN.

Sin duplicados

Cada elemento es único. Los repetidos se eliminan.

Sin orden

No mantienen el orden en que fueron declarados.

Elementos inmutables

Sus elementos deben ser inmutables (no listas ni dicts).

Crear un conjunto:

```
conjunto = set([5, 4, 6, 8, 8, 1])  
print(conjunto)          # {1, 4, 5, 6, 8} ← sin duplicados, sin orden  
print(type(conjunto))    # <class 'set'>  
  
# También: conjunto = {1, 2, 3}
```

Métodos de Conjuntos

`add(elem)`

Agrega un elemento.

```
c = {1, 2}
c.add(3) # {1, 2, 3}
```

`remove(elem)`

Elimina. Error si no existe.

```
c = {1, 2}
c.remove(2) # {1}
```

`discard(elem)`

Elimina. NO da error si no existe.

```
c = {1, 2}
c.discard(3) # {1, 2}
```

`union() o |`

Junta dos conjuntos.

```
a={1,2}; b={2,3}
a | b # {1, 2, 3}
```

`pop()`

Elimina un elemento aleatorio.

```
c = {1, 2}
c.pop() # aleatorio
```

`clear()`

Elimina todos los elementos.

```
c = {1, 2}
c.clear() # set()
```

Comparación: Lista vs Tupla vs Conjunto

Característica	Lista []	Tupla ()	Conjunto { }
Mutable	✓ Sí	✗ No	✓ Sí (add/remove)
Ordenada	✓ Sí	✓ Sí	✗ No
Duplicados	✓ Permite	✓ Permite	✗ No permite
Indexada	✓ Sí	✓ Sí	✗ No
Sintaxis	[1, 2, 3]	(1, 2, 3)	{ 1, 2, 3 }
Uso ideal	Datos que cambian	Datos fijos	Valores únicos



¿Cuándo usar cada una?

Lista: cuando necesitas modificar datos. Tupla: cuando los datos no deben cambiar (coordenadas, constantes). Conjunto: cuando necesitas eliminar duplicados o verificar pertenencia rápidamente.



Actividad Práctica: Tuplas y Conjuntos

1. Crea una tupla con 5 ciudades de Colombia e imprime la tercera
2. Intenta cambiar un elemento de la tupla. ¿Qué error obtienes?
3. Convierte la tupla a lista, agrega una ciudad, y conviértela de nuevo a tupla
4. Crea un conjunto con números del 1 al 10, incluyendo repetidos. ¿Qué pasa?
5. Crea dos conjuntos y haz su unión con el operador |



Tiempo: 10 minutos

Algoritmos de Ordenamiento

Son instrucciones que toman una lista y organizan sus elementos en un orden específico (ascendente o descendente, numérico o alfabético).



¿Por qué son importantes?

Reducen la complejidad de problemas. Se usan en búsquedas, bases de datos, algoritmos de división y conquista, y muchas más áreas de la informática.

Antes de elegir un algoritmo, pregúntate:

- ¿Cuán grande es la colección a ordenar?
- ¿Cuánta memoria está disponible?
- ¿La colección necesita expandirse?

Se clasifican por:

Intercambios • Comparaciones • Recursividad • Estabilidad • Espacio adicional

Algoritmo: Burbuja (BubbleSort)

Compara pares de elementos adyacentes y los intercambia si están en el orden incorrecto. Repite hasta que no haya más intercambios.

¿Cómo funciona?

Recorre la lista comparando cada elemento con el siguiente.
Si $n > n+1$, los intercambia.
Repite hasta que esté ordenada.

✓ Simple de entender

⚠ Lento con listas grandes

Ejemplo: $[5,3,8,1] \rightarrow [3,5,1,8] \rightarrow [3,1,5,8] \rightarrow [1,3,5,8]$

Algoritmo: Selección (SelectionSort)

Busca el elemento más pequeño en la parte no ordenada y lo coloca al inicio. Repite con el resto.

¿Cómo funciona?

Divide la lista en dos partes: ordenada y no ordenada.
Busca el mínimo en la parte no ordenada.
Lo intercambia con el primer elemento no ordenado.

✓ Pocos intercambios

⚠ Lento con listas grandes

Ejemplo: $[5, 3, 8, 1] \rightarrow [1, 3, 8, 5] \rightarrow [1, 3, 8, 5] \rightarrow [1, 3, 5, 8]$

Algoritmo: Inserción (InsertionSort)

Toma cada elemento y lo inserta en su posición correcta comparándolo con los anteriores.

¿Cómo funciona?

Recorre desde el segundo elemento.
Toma el actual y lo compara hacia atrás.
Lo inserta en la posición correcta.

✓ Bueno para listas casi ordenadas

⚠ Lento para listas desordenadas

Ejemplo: $[5, 3, 8, 1] \rightarrow [3, 5, 8, 1] \rightarrow [3, 5, 8, 1] \rightarrow [1, 3, 5, 8]$

Algoritmo: Shell

Mejora de InsertionSort. Compara elementos separados por un espacio (gap) que se reduce gradualmente.

¿Cómo funciona?

Usa un "gap" (espacio) grande al inicio.
Compara elementos separados por ese gap.
Reduce el gap hasta que sea 1.
Termina con inserción simple.

✓ Más rápido que inserción

⚠ Complejidad difícil de analizar

Ejemplo: Gap=4 → Gap=2 → Gap=1 → Ordenado

Algoritmo: Mezcla (MergeSort)

Divide la lista a la mitad recursivamente, ordena cada mitad, y luego las combina en una sola lista ordenada.

¿Cómo funciona?

Si la lista tiene 0 o 1 elemento, ya está ordenada.
Divide la lista en dos mitades.
Ordena cada mitad recursivamente.
Combina las dos mitades ordenadas.

✓ Siempre eficiente $O(n \log n)$

⚠ Requiere memoria adicional

Ejemplo: $[5, 3, 8, 1] \rightarrow [5, 3][8, 1] \rightarrow [3, 5][1, 8] \rightarrow [1, 3, 5, 8]$

Algoritmo: Rápido (QuickSort)

Elige un pivote, coloca los menores a la izquierda y los mayores a la derecha. Repite recursivamente.

¿Cómo funciona?

Elige un elemento como pivote.
Coloca menores a la izquierda del pivote.
Coloca mayores a la derecha.
Repite para cada sublista.

✓ Muy rápido en promedio

⚠ Peor caso $O(n^2)$

Ejemplo: Pivote=3: $[5,3,8,1] \rightarrow [1][3][5,8] \rightarrow [1,3,5,8]$

Comparación de Algoritmos de Ordenamiento

Algoritmo	Velocidad	Memoria	Estable
Burbuja	Lento $O(n^2)$	Poca	✓ Sí
Selección	Lento $O(n^2)$	Poca	✗ No
Inserción	Medio $O(n^2)$	Poca	✓ Sí
Shell	Medio	Poca	✗ No
MergeSort	Rápido $O(n \log n)$	Mucha	✓ Sí
QuickSort	Rápido $O(n \log n)$	Poca	✗ No

💡 En Python, el método `.sort()` usa TimSort, un algoritmo híbrido basado en MergeSort e InsertionSort, muy eficiente para datos del mundo real.



Resumen de la Clase



Estructuras de datos: formas organizadas de almacenar información



Listas []: ordenadas, mutables, indexadas, con muchos métodos útiles



Listas anidadas: listas dentro de listas para tablas y matrices



Tuplas (): como listas pero INMUTABLES — datos que no deben cambiar



Conjuntos { }: sin duplicados, sin orden — ideales para valores únicos



Algoritmos de ordenamiento: Burbuja, Selección, Inserción, Shell, Merge, Quick

¡Ahora tienes las herramientas para organizar datos como un profesional! 🚀

¿Preguntas?

*"Organizar bien tus datos es el primer paso
para resolver cualquier problema"*